





*Serviço Nacional
de Aprendizagem
Industrial*

Herança e Polimorfismo

Programação de Aplicativos

Herança e Polimorfismo

- **Instrutor:** Lucas Ribeiro Arêas
- **Unidade Curricular:** Programação de Aplicativos
- **Curso:** Técnico em Desenvolvimento de Sistemas

SENAI Vitória - DR/ES

1 - Herança

Herança

Uma das vantagens das linguagens orientadas a objeto é a possibilidade de se reutilizar código. Mas, quando falamos em reutilização de código, precisamos de recursos muito mais poderosos do que simplesmente copiar e alterar o código.

Herança

Um dos conceitos de orientação a objetos que possibilita a reutilização de código é o conceito de herança. Pelo conceito de herança é possível criar uma nova classe a partir de outra classe já existente.

Herança

Para que possamos entender melhor este conceito, vamos voltar ao nosso exemplo da classe **Conta** e criar uma nova classe que representará contas especiais do nosso sistema bancário online.

Herança

Essas novas contas permitirão o usuário realizar saques acima do saldo disponível, mas até um certo limite que será definido no código. Ou seja, o cliente poderá ficar com saldo negativo no banco até um certo limite.

Herança

Esta nova classe que representará as contas especiais vai se chamar **ContaEspecial** e será herdada a partir da classe **Conta** criada por nós nas aulas anteriores.

Herança

Ao ser criada a partir da classe **Conta**, a classe **ContaEspecial** conterá os métodos e atributos da classe herdada.

Herança

Além disso, a nova classe criada, que também podemos chamar de classe filha, derivada ou subclasse; pode ter seus próprios atributos e métodos.

Herança

Para que possamos fazer uma classe herdar de uma outra suas características, devemos utilizar a palavra reservada ***extends***, conforme veremos no exemplo de código a seguir.

Herança

```
public class ContaEspecial extends Conta {  
    private double limite;  
  
    public double getLimite() {  
        return limite;  
    }  
  
    public void setLimite(double limite) {  
        this.limite = limite;  
    }  
}
```

Herança

Para a situação que acabamos de analisar, podemos perceber que a classe **ContaEspecial** define um tipo mais especializado de conta.

Herança

Desta forma, podemos dizer que o mecanismo de criar novas classes herdando características de outras recebe o nome de *especialização*.

Herança

Bem, agora suponha que tenhamos um outro tipo de conta: a **ContaPoupanca**. A **ContaPoupanca** tem tudo o que a classe **Conta** tem com um método a mais que permite atribuir um reajuste percentual ao saldo.

Herança

Agora temos duas classes herdando da classe **Conta**. Nesse contexto podemos dizer que a classe **Conta** *generaliza* os conceitos de **ContaEspecial** e **ContaPoupanca**.

Herança

Vejam os a seguir o código da classe **ContaPoupanca** que herda as características da classe **Conta** e implementa um novo método: *reajustar()*.

Herança

```
public class ContaPoupanca extends Conta{  
    public void reajustar(double percentual){  
        double saldoAtual = this.getSaldo();//Obtenho o saldo atual da conta  
        double reajuste = saldoAtual * percentual;//Calculo o reajuste  
        this.depositar(reajuste);//deposito o reajuste na conta;  
    }  
}
```

Herança

Quando visualizamos uma hierarquia partindo da classe pai para filhas, dizemos que houve uma *especialização* da **superclasse**. Quando visualizamos partindo das classes filhas para as classes ancestrais, dizemos que houve uma *generalização* das **subclasses**.

Herança

Após definir as classes **ContaPoupanca** e **ContaEspecial**, conforme vimos anteriormente, o NetBeans nos sinalizará com erros nas duas classes pois, na aula anterior definimos construtores para a classe **Conta** que é ancestral das duas que criamos agora e não definimos construtores para as duas subclasses que acabamos de gerar.

Herança

Caso você não tenha definido um construtor em sua **superclasse**, não será obrigado a definir construtores para as **subclasses**, pois Java utilizará o construtor padrão para a **superclasse** e para as subclasses.

Herança

Porém, caso haja algum construtor definido na **superclasse**, obrigatoriamente você precisará criar ao menos um construtor para cada **subclasse**.

Herança

Vale ressaltar que os construtores das **subclasses** utilizarão os construtores das **superclasses** pelo uso da palavra reservada `super`.

Herança

A seguir veremos o código utilizado para criar o construtor da classe **ContaEspecial**.

Herança

```
public ContaEspecial(int numero, String nome_titular, double limite) {  
    super(numero, nome_titular);  
    this.limite = limite;  
}
```

Herança

O próximo código mostra o construtor da classe **ContaPoupanca**.

Herança

```
public ContaPoupanca(int numero, String nome_titular){  
    super(numero,nome_titular);  
}
```

Herança

Note que o construtor da classe **ContaEspecial** recebe como parâmetros o número, o nome do titular e o limite.

Herança

Então, pelo uso da palavra `super`, esse construtor “chama” o construtor da classe **Conta** (que criamos em aula anterior) repassando o número e o nome do titular e, depois, atribui ao limite o valor recebido como parâmetro.

Herança

Já o construtor da classe **ContaPoupanca** apenas se utiliza do construtor da superclasse, pois ele não recebe atributos além dos já tratados pela superclasse.

2 - Utilização de Atributos *Protected*

Utilização de Atributos Protected

Quando estudamos encapsulamento aprendemos que devemos preferencialmente manter os atributos com nível de acesso privado (*private*) de forma que para acessá-los outras classes precisem utilizar métodos.

Utilização de Atributos Protected

Mas, vimos também que há um nível de acesso protegido (*protected*) que faz com que o atributo se comporte como público para classes da mesma hierarquia ou do mesmo pacote e como privado para as demais classes.

Utilização de Atributos Protected

Definir alguns atributos da superclasse como `protected` pode trazer algumas facilidades para implementar métodos das subclasses.

Utilização de Atributos Protected

Por exemplo, observe que na implementação do método *reajustar* da classe

ContaPoupanca que vimos anteriormente, tivemos de obter o saldo, calcular o reajuste e depois depositar esse reajuste na conta para que fosse somado ao saldo, pois o atributo *saldo* foi definido na superclasse **Conta** como privado, não permitindo que o alterássemos diretamente de dentro da classe **ContaPoupanca**.

Utilização de Atributos Protected

O código a seguir apresenta uma nova implementação para o método *reajustar* da classe **ContaPoupanca**, considerando que o atributo saldo teve sua definição alterada para protegido (*protected*).

Utilização de Atributos Protected

```
public class ContaPoupanca extends Conta {  
  
    public void reajustar(double percentual) {  
        //Recalculo o saldo acessando diretamente o atributo  
        saldo = saldo + saldo * percentual;  
    }  
}
```

Utilização de Atributos Protected

Veja que na nova implementação podemos acessar diretamente o atributo, o que a torna mais simples.

Utilização de Atributos Protected

Apesar de potencialmente facilitar a implementação de métodos nas subclasses, a utilização de atributos protegidos é perigosa, pois o atributo ficará acessível a todas as classes que estejam no mesmo pacote e não somente às subclasses. Assim, pense bastante sobre as vantagens e desvantagens antes de se decidir por definir um atributo como *protected*.

3 - Polimorfismo

Polimorfismo

A palavra polimorfismo vem do grego *poli morfos* e significa muitas formas. Na orientação a objetos, isso representa uma característica que permite que classes diferentes sejam tratadas de uma mesma forma.

Polimorfismo

O polimorfismo permite escrever programas que processam objetos que compartilham a mesma superclasse (direta ou indiretamente) como se todos fossem objetos da superclasse; isso pode simplificar a programação.

Polimorfismo

Em outras palavras, podemos ver o polimorfismo como a possibilidade de um mesmo método ser executado de forma diferente de acordo com a classe do objeto que aciona o método e com os parâmetros passados para o método.

Polimorfismo

Com o polimorfismo podemos projetar e implementar sistemas que são facilmente extensíveis – novas classes podem ser adicionadas com pouca ou nenhuma alteração a partes gerais do programa, contanto que as novas classes façam parte da hierarquia de herança que o programa processa genericamente.

Polimorfismo

As únicas partes de um programa que devem ser alteradas para acomodar as novas classes são aquelas que exigem conhecimento direto das novas classes que adicionamos à hierarquia.

Polimorfismo

O polimorfismo pode ser obtido pela utilização dos conceitos de herança, sobrecarga de métodos e sobrescrita de método (também conhecida como redefinição ou reescrita de método).

4 - Sobrescrita

Sobrescrita

A técnica de **sobrescrita** permite reescrever um método em uma subclasse de forma que tenha comportamento diferente do método de mesma assinatura existente na sua superclasse.

Sobrescrita

Para ilustrar o conceito de sobrescrita, vamos criar um método *imprimirTipoConta()* na superclasse **Conta** e vamos sobrescrevê-lo nas duas subclasses da nossa hierarquia de exemplo (**ContaEspecial** e **ContaPoupanca**).

Sobrescrita

Esse método simplesmente imprimirá na tela uma mensagem de acordo com o tipo da conta, ou seja, de acordo com o tipo do objeto ele imprimirá uma mensagem diferente.

Sobrescrita

O código a seguir exibe apenas a linha de definição de cada classe e seu respectivo método *imprimirTipoConta()*.

Sobrescrita

```
public class Conta {  
  
    public void imprimirTipoConta() {  
        System.out.println("Conta Comum");  
    }  
}  
public class ContaEspecial extends Conta {  
  
    @Override  
    public void imprimirTipoConta() {  
        System.out.println("Conta Especial");  
    }  
}  
public class ContaPoupanca extends Conta {  
  
    @Override  
    public void imprimirTipoConta() {  
        System.out.println("Conta Poupança");  
    }  
}
```

Sobrescrita

Observe que no código que acabamos de ver, nas linhas anteriores aos métodos *ImprimirTipoConta* das classes **ContaEspecial** e **ContaPoupança**, há uma notação *@Override*.

Sobrescrita

A notação **@Override** é inserida automaticamente pelo NetBeans para indicar que esse método foi definido no ancestral e está sendo redefinido na classe atual.

Sobrescrita

A não colocação da notação **@Override** não gera erro, mas gera um aviso (*Warning*).

Sobrescrita

Isso ocorre porque entende-se que, quando lemos uma classe e seus métodos, é importante existir alguma forma de sabermos se um certo método foi ou não definido numa classe ancestral.

Sobrescrita

Assim a notação **@Override** é fundamental para aumentar a legibilidade e manutenibilidade do código.

Sobrescrita

Vejam os a seguir a utilização dos métodos definidos no código do slide 53.

Sobrescrita

```
package banco;
import java.util.Scanner;

public class UsaContaPolimorfa {
    public static void main(String[] args) {
        Conta c = null;
        Scanner scan = new Scanner(System.in);
        int opcao;
        System.out.println("Qual tipo de conta deseja criar para José?");
        System.out.println("1 - Conta");
        System.out.println("2 - Conta especial");
        System.out.println("3 - Conta poupança");
        opcao = scan.nextInt();
        switch (opcao) {
            case 1:
                c = new Conta(1, "José");
                break;
            case 2:
                c = new ContaEspecial(1, "José", 100.00);
                break;
            case 3:
                c = new ContaPoupanca(1, "José");
                break;
        }
        c.imprimirTipoConta();
    }
}
```

Sobrescrita

Vemos, nesse exemplo, que, de acordo com a opção escolhida no menu impresso, temos a criação de um objeto diferente.

Sobrescrita

Quando o usuário digita 2, por exemplo, é criada uma instância de **ContaEspecial**. Note que a variável **c** é do tipo **Conta**. Mas, ainda assim, é possível atribuir a ela uma instância de **ContaPoupanca** ou de **ContaEspecial** pois ambas herdam da classe **Conta**.

Sobrescrita

Na última linha do código apresentado, o método *imprimirTipoConta()* é chamado. Mas, sabemos que esse método foi implementado na classe **Conta** e sobrescrito nas duas classes filhas.

Sobrescrita

A resposta a essa pergunta vai depender da opção digitada pelo usuário! Por exemplo, caso o usuário digite 2, a variável **c** receberá uma instância de **ContaEspecial**. Nesse caso, na última linha será chamado o método *imprimirTipoConta()* da classe **ContaEspecial**.

Sobrescrita

Analogamente, caso o usuário digite a opção 1, será utilizado o método da classe **Conta** e, caso digite 3, será utilizado o método da classe **ContaPoupanca**.

Sobrescrita

Nesse exemplo que acabamos de ver, a mesma linha de código pode ter um comportamento diferente, dependendo das circunstâncias. Isso é polimorfismo!

Sobrescrita

Agora que entendemos o conceito de sobrescrita, vamos corrigir uma falha que cometemos ao definir nossa hierarquia de classes! A classe **ContaEspecial** tem um atributo limite que define um valor que o proprietário da conta poderia sacar mesmo não tendo saldo.

Sobrescrita

O problema é que quando implementamos a classe **ContaEspecial** não reescrevemos o método *sacar*; logo, essa classe está utilizando o método da superclasse **Conta**.

Sobrescrita

Assim, independentemente do valor do atributo limite da **ContaEspecial**, o saque não será efetuado caso não haja saldo suficiente, pois essa é a lógica implementada no método *sacar* da classe **Conta**.

Sobrescrita

O código a seguir exhibe uma implementação para o método *sacar* na classe **ContaEspecial** que sobrescreve o método da superclasse e permite a realização do saque caso o valor a ser sacado seja menor ou igual a soma entre o saldo e o limite da conta.

Sobrescrita

```
@Override
public boolean sacar(double valor){
    if (valor <= this.limite + this.saldo) {
        this.saldo -= valor;
        return true;
    } else {
        return false;
    }
}
```

5 - Sobrecarga

Sobrecarga

Métodos de mesmo nome podem ser declarados na mesma classe, contanto que tenham diferentes conjuntos de parâmetros (determinado pelo número, tipos e ordem dos parâmetros). Isso é chamado sobrecarga de método.

Sobrecarga

Para que os métodos de mesmo nome possam ser distinguidos, eles devem possuir assinaturas diferentes. A assinatura (*signature*) de um método é composta pelo nome do método e por uma lista que indica os tipos de todos os seus argumentos.

Sobrecarga

Assim, métodos com mesmo nome são considerados diferentes se recebem um diferente número de argumentos ou tipos diferentes de argumentos e têm, portanto, uma assinatura diferente.

Sobrecarga

Quando um método sobrecarregado é chamado, o compilador Java seleciona o método adequado examinando o número, os tipos e a ordem dos argumentos na chamada.

Sobrecarga

Mesmo sem saber, nós já utilizamos o conceito de sobrecarga quando criamos os construtores da classe **Conta**. Ao criar a classe **Conta** nós definimos dois construtores, sendo que um deles recebendo três parâmetros e o outro recebendo dois parâmetros.

Sobrecarga

Para ilustrar melhor o conceito de sobrecarga, implementaremos na classe **Conta** um novo método *imprimirTipoConta* que receberá como parâmetro uma *String* e imprimirá na tela o tipo da conta seguido pela *String* recebida.

Sobrecarga

O código a seguir exhibe os dois métodos *imprimirTipoConta* da classe **Conta**.

Sobrecarga

```
public void imprimirTipoConta() {  
    System.out.println("Conta Comum");  
}  
public void imprimirTipoConta(String s) {  
    System.out.println("Conta Comum - String recebida:" + s);  
}
```


Sobrecarga

O exemplo a seguir mostra o comportamento dos objetos diante o uso de herança, sobrecarga e sobrescrita.

Sobrecarga

Nesse exemplo, vemos tanto o código-fonte de uma classe que utiliza as classes **Conta** e **ContaEspecial** quanto a saída da execução desse programa (destacada em vermelho).

Sobrecarga

```
1  package banco;
2
3  public class UsaSobrecargaSobrescrita {
4
5      public static void main(String[] args) {
6          Conta c1 = new Conta(1, "Ze");
7          ContaEspecial c2 = new ContaEspecial(2, "João", 100);
8          c1.imprimirTipoConta();
9          c1.imprimirTipoConta("Teste Sobrecarga");
10         c2.imprimirTipoConta();
11         c2.imprimirTipoConta("Teste Sobrecarga em subclasse");
12     }
13 }
14
```

Saída - Aula4 (run)

```
run:
Conta Comum
Conta Comum - String recebida:Teste Sobrecarga
Conta Especial
Conta Comum - String recebida:Teste Sobrecarga em subclasse
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Sobrecarga

No código que acabamos de ver a variável **c1** contém uma instância da classe **Conta** e **c2**, uma instância da classe **ContaEspecial**. Após a criação dos objetos, primeiro é feita uma chamada ao método *imprimirTipoConta* de **c1** sem passar nenhum parâmetro, e depois outra chamada passando uma *String*.

Sobrecarga

Em seguida foram feitas as mesmas chamadas a partir de **c2**. Note que como não criamos na classe **ContaEspecial**, um método *imprimirTipoConta* que receba uma *String* como argumento, a última chamada feita a partir de **c2** foi atendida pelo método da classe **Conta**.

Sobrecarga

O que aconteceu foi que, como **c2** contém uma instância de **ContaEspecial**, o compilador procurou por um método *imprimirTipoConta (String)* na classe **ContaEspecial**. Como não encontrou, então o compilador fez uso do método existente na superclasse, já que **ContaEspecial** herda de **Conta**.

6 - Classe *Object*

Classe *Object*

Todas as classes no Java herdam direta ou indiretamente da classe ***Object***; portanto, seus 11 métodos são herdados por todas as outras classes.

Classe *Object*

Vejamos alguns métodos de exemplo.

Classe *Object*

toString(): esse método indica como transformar um objeto de uma classe em uma *String*. Ele é utilizado, automaticamente, sempre que é necessário transformar um objeto de uma classe em uma *String*. Na classe **Conta**, por exemplo, esse método poderia ser definido da seguinte forma:

Classe *Object*

@Override

```
public String toString() {  
    return ("Conta: " + this.numero);  
}
```

Classe *Object*

Note o uso da notação **@Override** para o método *toString()*. Isso ocorre porque estamos sobrescrevendo um método definido em ***Object***.

Classe *Object*

getClass: retorna a classe de um objeto. Muito utilizado quando se trabalha na criação de ferramentas geradoras de código ou *frameworks*. Este método será utilizado a seguir para construir o método *toString*.

Classe *Object*

equals(): esse método possibilita comparar os valores de dois objetos. Se considerarmos esses objetos iguais, devemos retornar *true*; caso sejam diferentes, devemos retornar *false*.

Classe *Object*

Quando comparamos dois objetos com o operador `==`, na realidade estamos comparando se eles são o mesmo objeto e não se seus valores são iguais. Isso ocorre porque os objetos em Java são ponteiros para espaços de memória.

Classe *Object*

Assim, dois objetos podem ter os mesmos valores em seus atributos e não serem iguais, pois podem apontar para locais diferentes. Dessa forma, para comparar os valores de dois objetos, devemos utilizar o método *equals*.

Classe *Object*

Por isso que quando queremos comparar Strings, por exemplo, utilizamos o método *equals*. O código a seguir exibe uma implementação do método *equals* para a classe **Conta**. Nesse método consideramos que duas contas são iguais se são de uma mesma classe e se têm o mesmo número.

Classe *Object*

```
@Override
public boolean equals(Object o) {
    if (o == null) {
        return false;
    } else if (o.getClass() != this.getClass()) {
        return false;

    } else if (((Conta) o).getNumero() != this.getNumero()) {
        return false;

    } else {
        return true;
    }
}
```

Classe *Object*

No método *equals* apresentado são feitas as seguintes verificações:

Classe *Object*

if (o == null): estamos prevendo que se pode tentar comparar um objeto **Conta** com um valor nulo (variável não instanciada). Como o objeto **Conta** que acionou o método *equals* está instanciado, ele não pode ser igual a *null*.

Classe *Object*

if (o.getClass() != this.getClass()): estamos verificando se o objeto passado como parâmetro é da mesma classe que o objeto que está invocando o método, ou seja, se estamos comparando duas instâncias da classe **Conta**. Caso os objetos sejam de classes diferentes, consideramos que eles são diferentes.

Classe *Object*

if (((Conta) o).getNumero() != this.getNumero()): caso os dois objetos sejam do mesmo tipo (**Conta**), então comparamos os valores do atributo *numero* dessas contas. Se os números são diferentes, consideramos que são contas diferentes; caso contrário, as consideramos iguais.

Classe *Object*

É importante conhecermos a hierarquia de uma classe para evitarmos replicar códigos de forma desnecessária.

Obrigado!